

Prática de Laboratório

Disciplina:
Construção de
Compiladores

Professor:
Robson Lins

Objetivos

- Compreender o funcionamento de um analisador sintático descendente recursivo.
- Relacionar uma gramática livre de contexto aos procedimentos de um programa.
- Implementar procedimentos recursivos em C.
- Reconhecer expressões sintaticamente válidas e inválidas.
- Identificar e tratar erros sintáticos.

Contextualização

1

O analisador sintático verifica se os símbolos estão organizados de acordo com as regras da gramática.

2

Nesta prática será utilizado o modelo de analisador descendente recursivo.

3

Cada não terminal da gramática será associado a um procedimento/função.

4

Fluxo: entrada → tokens → analisador sintático → expressão válida ou erro.

Gramática da prática

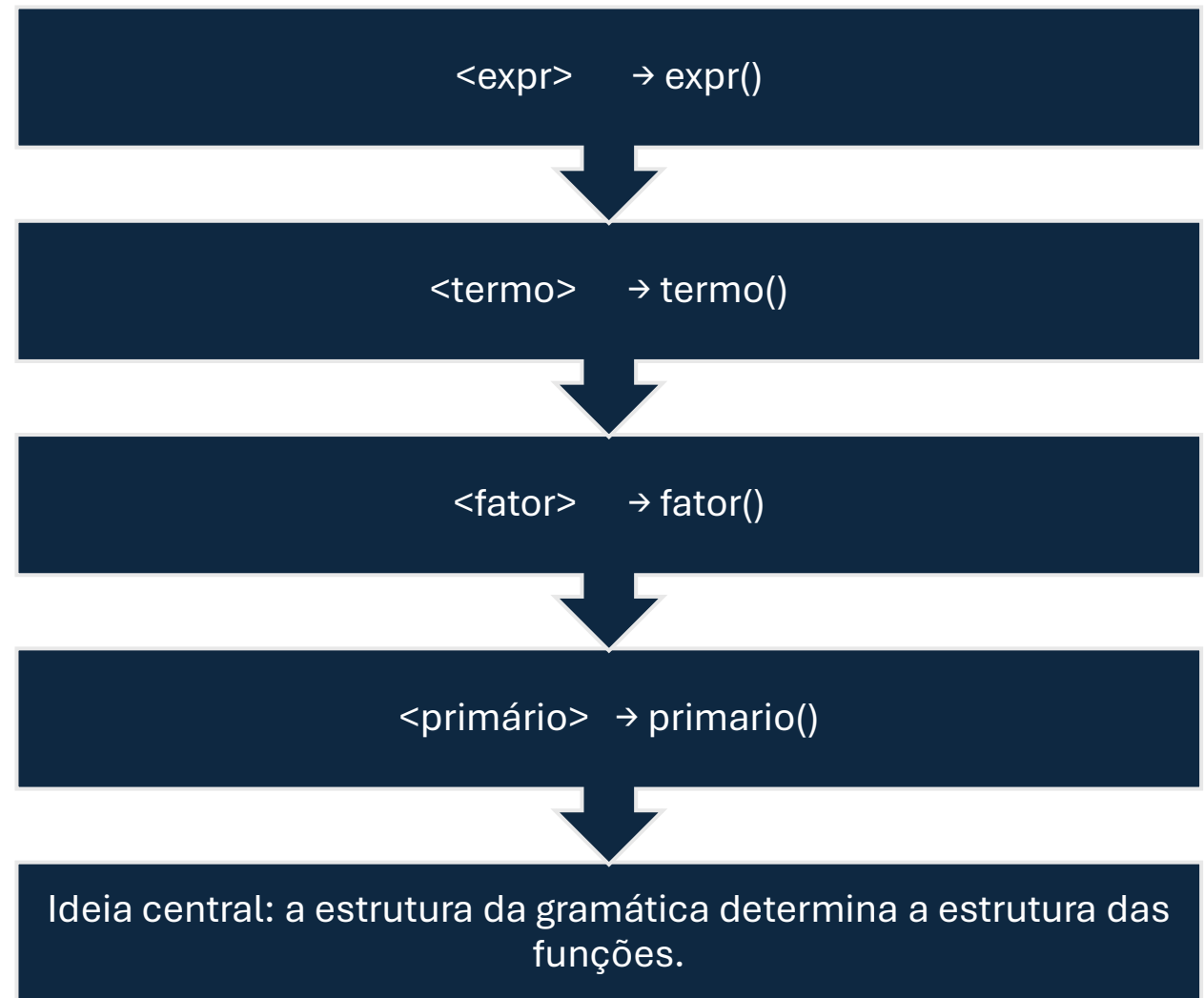
$\langle \text{expr} \rangle ::= \langle \text{termo} \rangle + \langle \text{expr} \rangle \mid \langle \text{termo} \rangle$

$\langle \text{termo} \rangle ::= \langle \text{fator} \rangle * \langle \text{termo} \rangle \mid \langle \text{fator} \rangle$

$\langle \text{fator} \rangle ::= \langle \text{primário} \rangle ** \langle \text{fator} \rangle \mid \langle \text{primário} \rangle$

$\langle \text{primário} \rangle ::= \text{IDENT} \mid \text{NÚMERO} \mid (\langle \text{expr} \rangle)$

Da gramática para a linguagem C



Problema da prática



Desenvolva, em C, um analisador sintático descendente recursivo.



O programa deve verificar se uma sequência de tokens pertence à linguagem definida pela gramática.



Implemente: `expr()`, `termo()`, `fator()` e `primario()`.



Implemente também as rotinas auxiliares necessárias.

Algoritmo principal

```
procedimento ANALISADOR_SINTATICO
início
    obtenha_símbolo()
    EXPR()

    se símbolo_lido = FIM então
        escreva "Expressão válida"
    senão
        ERRO("símbolo inesperado")
fim
```

Procedimento EXPR

```
procedimento EXPR
início
    TERMO()

    se símbolo_lido = '+' então
        início
            obtenha_símbolo()
            EXPR()
        fim
    fim
fim
```


Procedimento TERMO

```
procedimento TERMO
início
    FATOR()

    se símbolo_lido = '*' então
        início
            obtenha_símbolo()
            TERMO()
        fim
    fim
fim
```

Procedimento FATOR

```
procedimento FATOR
início
    PRIMARIO()

    se símbolo_lido = '**' então
        início
            obtenha_símbolo()
            FATOR()
        fim
    fim
fim
```

Procedimento PRIMARIO

```
procedimento PRIMARIO
início
    se símbolo_lido = IDENT então
        obtenha_símbolo()

    senão se símbolo_lido = NUMERO então
        obtenha_símbolo()

    senão se símbolo_lido = '(' então
        início
            obtenha_símbolo()
            EXPR()

            se símbolo_lido ≠ ')' então
                ERRO("falta ')'")
            senão
                obtenha_símbolo()
        fim

    fim

    senão
        ERRO("primário inválido")
fim
```

Estrutura inicial em linguagem C

```
#include <stdio.h>
#include <stdlib.h>
```

```
#define IDENT      1
#define NUMERO     2
#define MAIS       3
#define MULT       4
#define POTENCIA   5
#define ABRE_PAR   6
#define FECHA_PAR  7
#define FIM        8
```

```
int simbolo_lido;
```

```
void obtenha_simbolo(void);
void erro(const char *mensagem);
```

```
void expr(void);
void termo(void);
void fator(void);
void primario(void);
```

Tarefa de implementação



Traduza cada procedimento do pseudocódigo para uma função C.



Implemente `obtenha_simbolo()` para avançar pela sequência de tokens.



Implemente `erro()` para informar erros sintáticos.



Mantenha a variável `simbolo_lido` como símbolo corrente.



Não altere a gramática: altere apenas a implementação.

Representação da entrada

Para concentrar a prática na análise sintática, a entrada será uma sequência de tokens.



Exemplo textual: IDENT + NUMERO * IDENT



Representação possível: IDENT, MAIS, NUMERO, MULT, IDENT, FIM.



Cada chamada de `obtenha_simbolo()` deve avançar para o próximo token.

Testes — expressões válidas

- 1. IDENT
- 2. IDENT + NUMERO
- 3. IDENT * NUMERO
- 4. IDENT ** NUMERO
- 5. (IDENT + NUMERO)
- 6. IDENT + NUMERO * IDENT

Testes — expressões inválidas

- 7. IDENT +
- 8. (IDENT + NUMERO
- 9. IDENT * + NUMERO
- 10. + IDENT
- O programa deve informar que existe erro sintático.
- No caso 8, deve identificar a ausência do ')'

Registro dos testes

Para cada entrada, registre: resultado obtido, resultado esperado e se o teste foi aprovado.

Compare sistematicamente os resultados do programa com os resultados esperados.

Use os testes para localizar erros nas funções recursivas.

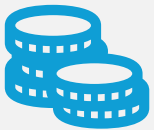
Desafio adicional



Modifique o programa para mostrar o caminho percorrido pelo analisador.



Mostre as chamadas:
EXPR → TERMO → FATOR
→ PRIMARIO.



Indique os tokens reconhecidos durante o percurso.



Objetivo: visualizar a recursividade e a relação entre gramática e execução.

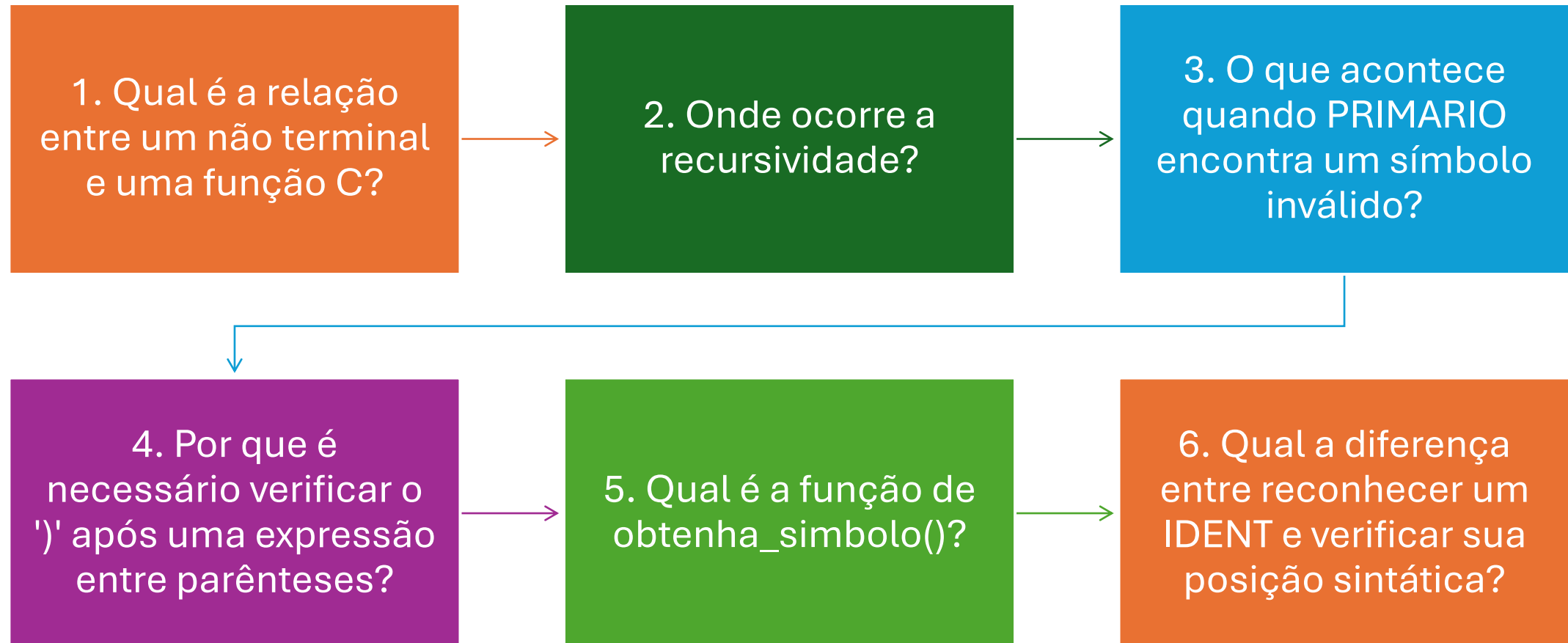
Exemplo de rastreamento

Entrada:
IDENT + NUMERO

```
EXPR
  TERMO
    FATOR
      PRIMARIO -> IDENT
    +
  EXPR
    TERMO
      FATOR
        PRIMARIO -> NUMERO
```

EXPRESSAO VALIDA

Perguntas de reflexão



Entrega

Código-fonte em C.

Tabela dos testes realizados.

Respostas às perguntas de reflexão.

Implementação do desafio adicional, se solicitado.

Verifique se o código compila e se todos os testes foram executados.

Fechamento



Gramática → pseudocódigo →
funções C → execução →
reconhecimento sintático



O ponto central da prática é
perceber como as regras da
gramática se transformam em
procedimentos recursivos.



Próxima evolução: integrar o
analisador sintático ao
analisador léxico.